

Parallel and distributed Computing

Lecture 12

Bazam-e-Jahan

Lecturer

The University of Faisalabad

- Loosely synchronized clocks can facilitate checkpoint coordination.
- Loosely synchronized clocks can trigger the local checkpointing actions of all participating processes at approximately the same time without a checkpoint initiator.
- A process takes a checkpoint and waits for a period that equals the sum of the maximum deviation between clocks and the maximum time to detect a failure in another process in the system.
- The process can be assured that all checkpoints belonging to the same coordination session have been taken without the need of exchanging any messages.

Minimal checkpoint coordination

- Coordinated checkpointing requires all processes to participate in every checkpoint. This approach is not scalable.
- Basic Idea: Only those processes which communicated with the initiator directly or indirectly since the last checkpoint need to take new checkpoints.

Minimal checkpoint coordination (contd.)

- Two Phase Protocol:
- Phase 1:
 - Initiator sends a checkpoint request to all the processes which communicated with it since the last checkpoint.
 - Each process (P_i) which received this request forwards it to all the processes which communicated with (P_i) since the last checkpoint, and so on until no more processes can be identified.
- Phase 2:
 - All processes identified in the first phase take a checkpoint

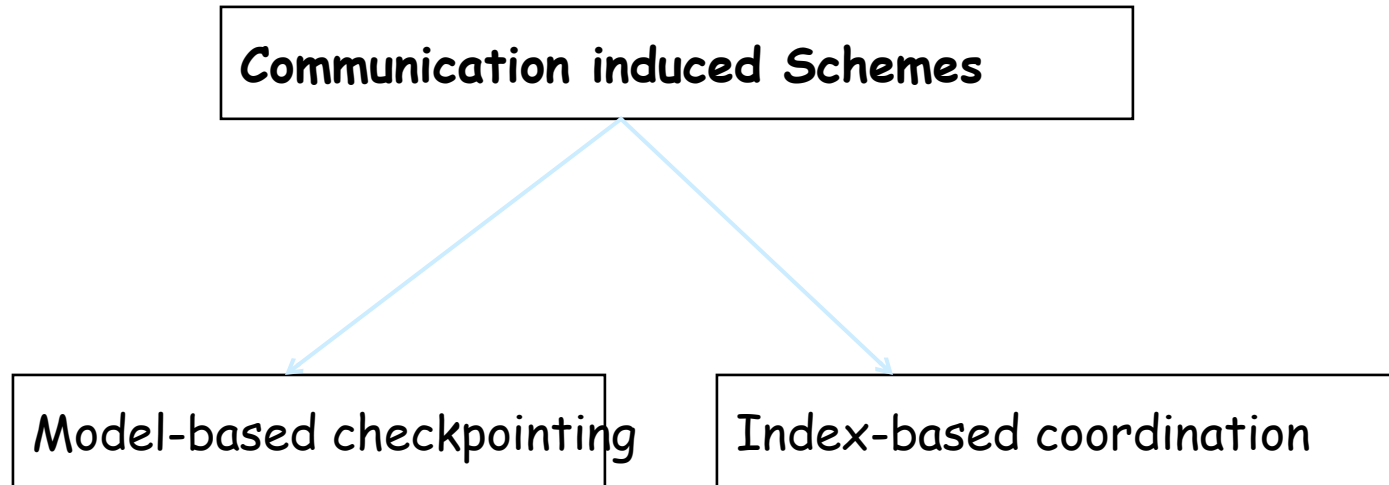
Communication-induced checkpointing

- Avoids the domino effect while allowing processes to take some of their checkpoints independently.
- However, process independence is constrained to guarantee the eventual progress of the recovery line, and therefore processes may be forced to take additional checkpoints.
- The checkpoints that a process takes independently are **local checkpoints** while those that a process is forced to take are called **forced checkpoints**.

Communication-induced checkpoint (contd.)

- Protocol related information is piggybacked to the application messages.
- Receiver of each application message uses the piggybacked information to determine if it has to take a forced checkpoint to advance the global recovery line.
- The forced checkpoint must be taken before the application may process the contents of the message, possibly incurring high latency and overhead.
- Therefore, reducing the number of forced checkpoints is important.
- No special coordination messages are exchanged.

Communication-induced Checkpointing schemes



Model based checkpointing



- Model-based checkpointing relies on preventing patterns of communications and checkpoints that could result in inconsistent states among the existing checkpoints.
- A model is set up to detect the possibility that such patterns could be forming within the system, according to some heuristic.
- A checkpoint is usually forced to prevent the undesirable patterns from occurring.

Index-based communication- induced checkpointing

- Index-based checkpointing works by assigning monotonically increasing indexes to checkpoints, such that the checkpoints having the same index at different processes form a consistent state.
- The indices are piggybacked on application messages to help receivers decide when they should force a checkpoint.

- It combines checkpointing with logging of nondeterministic events.
- It relies on the piecewise deterministic (PWD) assumption, which postulates that all nondeterministic events that a process executes can be identified and that the information necessary to replay each event during recovery can be logged in the event's determinant (all info. necessary to replay the event).
- By logging and replaying the nondeterministic events in their exact original order, a process can deterministically recreate its pre-failure state even if this state has not been checkpointed.
- Log-based rollback recovery is in general attractive for applications that frequently interact with the outside world which consists of input and output logged to stable storage.

- Schemes differ in the way the determinants are logged into the stable storage.
- Pessimistic Logging: The application has to block waiting for the determinant of each nondeterministic event to be stored on stable storage before the effects of that event can be seen by other processes or the outside world. It simplifies recovery but hurts the failure-free performance.
- Optimistic Logging: The application does not block, and the determinants are spooled to stable storage asynchronously. It reduces failure free overhead, but complicates recovery.
- Casual Logging: Low failure free overhead and simpler recovery are combined by striking a balance between optimistic and pessimistic logging.

Kindly read the book before and after the lecture, if some terms/ words are not covered in the lecture, if appear in exam it might not appear strange or out of course.